

UNIVERSITY OF CLAUDE · COURSE SYLLABUS

CSCE 608 — Database Systems

How a database is actually built — storage, indexing, query execution, concurrency, and recovery

AT A GLANCE

Credits: 3 credit-equivalents · **Modules:** 13 · **Term:** Semester 3 (with CSCE 647 and CSCE 649)

Prerequisites: CSCE 629 Analysis of Algorithms; CSCE 611 Operating Systems helpful; fluency in one systems language

Effort: 10–12 hours per week · 13 modules + 2 project weeks

Deliverable: A working storage engine: buffer pool, B+tree, query executor, concurrency control, and write-ahead logging — with crash recovery you have tested by actually killing the process

Course description

Most courses teach you to *use* a database. This one teaches you to build one, which is a different subject and a more useful one — because once you know what happens underneath, the behaviour of a real system stops being folklore.

The through-line is a single question: **how do you make a machine that loses power behave as though it did not?** Almost every design decision in a database descends from that. Durability is why there is a log; the log's cost is why there is a buffer pool; concurrent access to the buffer pool is why there is locking; and the cost of locking is why there is multi-version concurrency control.

The second theme is that **the storage hierarchy dictates the algorithms**. A B+tree is not a better binary tree; it is the data structure you get when a random read costs ten thousand times a sequential one. Change that ratio — as SSDs did — and the right answers change with it, which is exactly what has happened to this field over the last fifteen years.

Everything is implemented. By the end you will have a storage engine that survives kill -9 in the middle of a transaction, and you will be able to explain exactly why.

Learning outcomes

On finishing this course you should be able to do the following without reference material.

- 1 Explain how a DBMS lays out data on disk and why.
- 2 Implement a buffer pool with a replacement policy and pin counts.

- 3 Implement a B+tree with correct splits, merges, and concurrent access.
- 4 Explain the relational algebra behind SQL and the operators that implement it.
- 5 Implement join algorithms and analyse their I/O cost.
- 6 Explain cost-based query optimisation and why it goes wrong.
- 7 State the ACID properties precisely and say what each costs.
- 8 Implement two-phase locking and explain how MVCC avoids it.
- 9 Implement write-ahead logging and ARIES-style recovery.
- 10 Explain the trade-offs that distributed databases actually face.

Module schedule

One module per week for thirteen weeks, then two weeks for the final project. Each module is a slide deck, a set of lecture notes, mapped external lectures, and exercises that must run.

Wk	Module	Core question it answers
1	What a Database Is For	Why not just use files?
2	Storage and the Buffer Pool	How does a database decide what to keep in RAM?
3	B+Trees	Why is every database index the same shape?
4	Hash Indexes, LSM Trees, and Concurrent Access	When is a B+tree the wrong answer?
5	Relational Algebra and SQL	What does a SQL query actually say?
6	Sorting and Joins	How do you sort or join data larger than memory?
7	Query Execution	How does a query plan actually produce rows?
8	Query Optimisation	How does a database decide how to run your query?
9	Transactions and Concurrency Control	What does a transaction guarantee, and how?
10	Multi-Version Concurrency Control	How do you let readers and writers run at the same time?
11	Logging and Recovery	How do you survive losing power mid-transaction?
12	Column Stores and Analytics	Why are analytical databases built differently?
13	Distributed Databases	What actually changes when the data is on many machines?

Projects

Project 1 — A storage engine (after Module 7)

The bottom half of a database. Everything after this depends on it, so correctness matters more than speed — though you will measure both.

Requirements

- A page-based disk manager with a slotted page layout supporting variable-length records.
- A buffer pool with LRU-K or CLOCK replacement, pin counts, and dirty-page tracking.
- A B+tree index supporting insert, delete, point lookup, and range scan, with correct splits and merges.
- Latch-crabbing for concurrent B+tree access.
- A sequential scan and an index scan operator.
- Instrumentation: page reads, page writes, buffer hit rate, and tree height, all reported.

Done means

- **A million-key B+tree that survives a randomised workload of inserts, deletes, and range scans,** verified against an in-memory reference map after every operation.
- A buffer hit rate plot against pool size for a Zipfian workload. **Explain the shape of the curve.**
- Measured page I/O for a range scan against the analytic prediction from the tree height and fanout.
- **Eight threads hammering the tree concurrently with no corruption and no deadlock,** for ten million operations.

Project 2 — Transactions and recovery (after Module 12)

The top half. This is where a database stops being a data structure and becomes a database.

Requirements

- Two-phase locking with deadlock detection or prevention, and the deadlock rate reported.
- Write-ahead logging with log sequence numbers and the WAL protocol correctly enforced.
- ARIES-style recovery: analysis, redo, undo.
- Checkpointing, with the recovery time measured as a function of checkpoint interval.
- **Either** MVCC with snapshot isolation **or** serializable 2PL — with the isolation level you provide stated precisely and demonstrated.

Done means

- **A crash test that kills the process at a thousand random points during a transactional workload, and recovers to a consistent state every time.** Report the thousand results, not a representative one.

- A demonstration of each anomaly your isolation level permits and each one it prevents, with the SQL that exhibits it.
- Throughput against concurrency level, showing where lock contention begins to dominate.
- **An honest failure analysis:** name a crash point your recovery would not survive, or prove there is none.

Assessment

Nobody grades this. The standard is therefore mechanical, and you apply it to yourself honestly or the credential is worth nothing.

Component	Weight	Standard
Module exercises	40%	Code runs and produces the specified output. Not 'I understand how I would do it.'
Projects	45%	In the portfolio repository, with a README showing results and a written note on what was hard.
Self-checks	15%	Answered in writing from memory. Recognition is not knowledge; writing exposes the difference.

Required materials

All free. Nothing in this course requires a purchase.

Resource	Role
CMU 15-445/645 Database Systems — Andy Pavlo (free video, slides, projects)	The primary source. Complete lectures, slides, and the BusTub project assignments. The best free database systems course that exists, by some margin.
CMU 15-721 Advanced Database Systems (free)	Where modern in-memory and column-store systems are covered. Watch the relevant lectures after Modules 05 and 09.
MIT 6.5830 Database Systems (free)	A second voice with a different emphasis — more on query processing theory and less on implementation detail.
The Red Book — Readings in Database Systems , 5th ed. (free)	Curated classic papers with editorial commentary explaining why each mattered. Read the commentary even where you skip the paper.
Database Internals — Alex Petrov (sample chapters free)	Strong on storage engines and on distributed systems. Good complement to the CMU material for Modules 02–04 and 13.
Architecture of a Database System — Hellerstein, Stonebraker & Hamilton (free)	A 100-page tour of how the components fit together. Read it in week one and again at the end.

Tooling

- **C++17 or Rust.** You are managing memory, page layouts, and latches; a garbage-collected language hides exactly what this course is about.
- **Raw files, not a filesystem abstraction.** You need control over when bytes reach the disk, which means `pwrite` and `fsync` and knowing what each guarantees.
- **SQLite installed as a reference.** Its source is famously readable and it implements almost everything in this course in about 150k lines.
- **PostgreSQL** for comparison, and for `EXPLAIN ANALYZE` — the single best tool for understanding query optimisation.
- **A crash-testing harness** that kills your process at a randomly chosen point and verifies recovery. Build it in Module 11; it is the only honest test of durability.
- **A benchmark:** YCSB or TPC-C style, so claims about performance are measured rather than asserted.

Academic honesty, restated for a solo student

There is no one to cheat but yourself, which makes it easier, not harder. Two rules: copy code only after you have written a worse version yourself, and never let a working result stand in for an understood one. When you paste something you do not understand, write a `TODO` naming what you do not understand, and go back to it.